

National University of Technology

Machine Learning - CS3503

LAB Submitted to:

Lec.Hajra Ahmad

Submitted By:

Muhammad Abdullah

Reg. No: **F22607010**
Batch: **Fall AI 2022**
Assignment : **01**

Q.NO.1:

```
import numpy as np
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder

data = {
    'age': ['<=30', '<=30', '>40', '>40', '>40', '31..40', '<=30', '<=30',
'31..40', '>40', '>40', '31..40'],
    'income': ['high', 'high', 'medium', 'low', 'low', 'low', 'medium',
'low', 'high', 'medium', 'medium', 'low'],
    'student': ['no', 'no', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'yes',
'no', 'no', 'no'],
    'credit_rating': ['fair', 'excellent', 'fair', 'fair', 'excellent',
'excellent', 'fair', 'fair', 'fair', 'excellent', 'fair', 'fair'],
    'buys_computer': ['no', 'no', 'yes', 'yes', 'no', 'yes', 'no', 'yes',
'yes', 'no', 'yes', 'no']
}

df = pd.DataFrame(data)
df.sample(10)
```

	age	income	student	credit_rating	buys_computer
1	<=30	high	no	excellent	no
11	31..40	low	no	fair	no
10	>40	medium	no	fair	yes
9	>40	medium	no	excellent	no
7	<=30	low	yes	fair	yes
4	>40	low	yes	excellent	no
5	31..40	low	yes	excellent	yes
8	31..40	high	yes	fair	yes
3	>40	low	yes	fair	yes
0	<=30	high	no	fair	no

```
# Encoding categorical features to numerical values
le = LabelEncoder()

df['age'] = le.fit_transform(df['age'])
df['income'] = le.fit_transform(df['income'])
df['student'] = le.fit_transform(df['student'])
df['credit_rating'] = le.fit_transform(df['credit_rating'])
df['buys_computer'] = le.fit_transform(df['buys_computer'])


X = df[['age', 'income', 'student', 'credit_rating']]
y = df['buys_computer']


nb_classifier = GaussianNB()
nb_classifier.fit(X, y)


customer_A = np.array([[1, 0, 0, 0]])
prediction_A = nb_classifier.predict(customer_A)
print("Customer A buys a computer?" , "Yes" if prediction_A[0] == 1 else
"No")

customer_B = np.array([[0, 2, 1, 1]])
prediction_B = nb_classifier.predict(customer_B)
print("Customer B buys a computer?" , "Yes" if prediction_B[0] == 1 else
"No")


customer_C = np.array([[2, 2, 1, 0]])
prediction_C = nb_classifier.predict(customer_C)
print("Customer C buys a computer?" , "Yes" if prediction_C[0] == 1 else
"No")


customer_D = np.array([[1, 0, 1, 1]])
prediction_D = nb_classifier.predict(customer_D)
print("Customer D buys a computer?" , "Yes" if prediction_D[0] == 1 else
"No")
```

```
Customer A buys a computer? No
Customer B buys a computer? Yes
Customer C buys a computer? Yes
Customer D buys a computer? Yes
```

```python
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Confusion matrix as given
confusion_matrix = np.array([[7, 6, 5],
                             [1, 2, 1],
                             [0, 0, 3]])

total_instances = np.sum(confusion_matrix)

correct_predictions = np.trace(confusion_matrix)

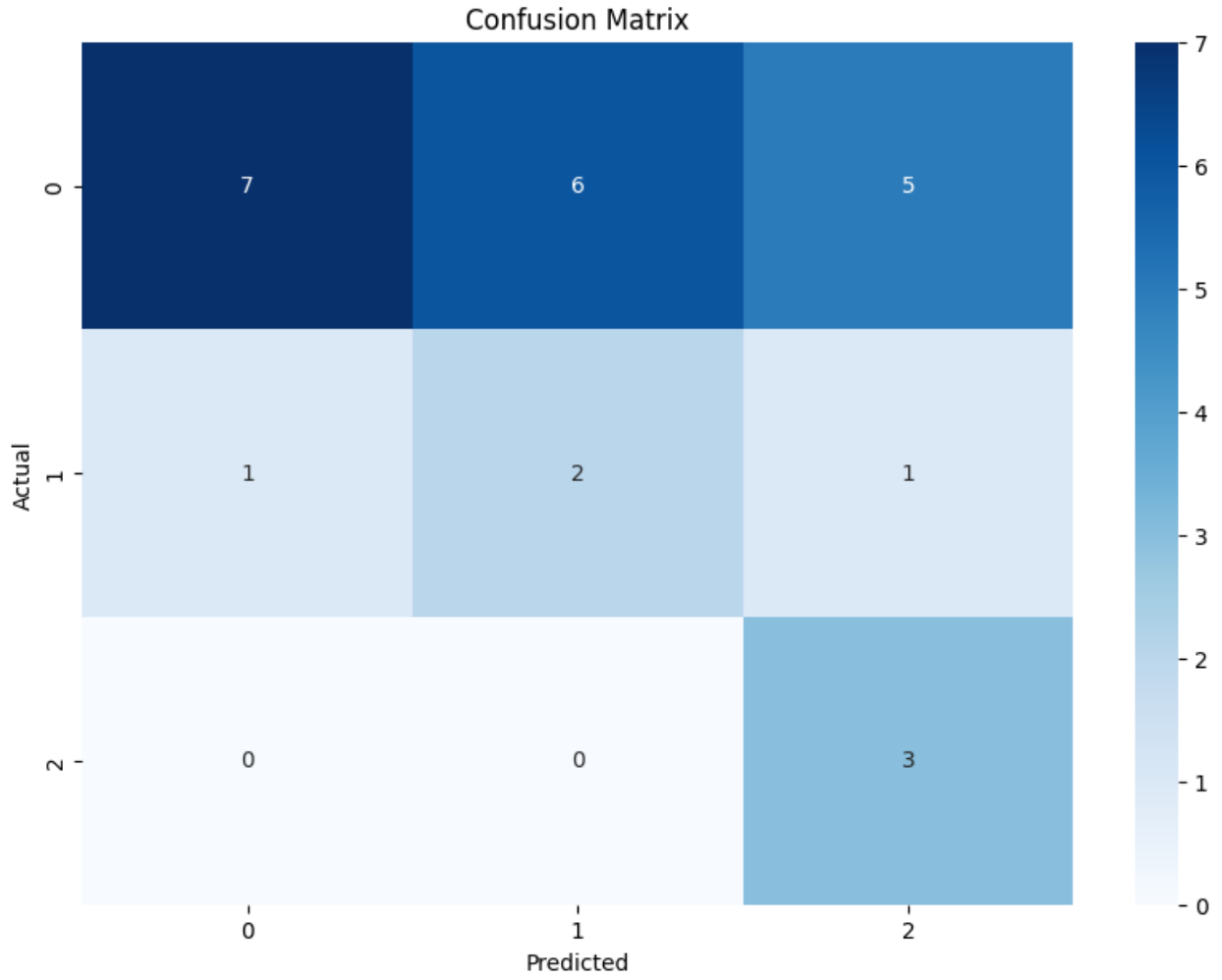
accuracy = correct_predictions / total_instances

error_rate = 1 - accuracy

print("Accuracy: ", accuracy)
print("Error Rate: ", error_rate)


plt.figure(figsize=(10, 7))
sns.heatmap(confusion_matrix, annot=True, cmap='Blues', fmt='g')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show
```

```
Accuracy:  0.48
Error Rate:  0.52
```




```
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree


df['age'] = df['age'].astype('category').cat.codes
df['income'] = df['income'].astype('category').cat.codes
df['student'] = df['student'].astype('category').cat.codes
df['credit_rating'] = df['credit_rating'].astype('category').cat.codes
df['buys_computer'] = df['buys_computer'].astype('category').cat.codes


X = df.drop(columns=['buys_computer'])
y = df['buys_computer']
```

```
clf = DecisionTreeClassifier(criterion='entropy')


clf.fit(X, y)

tree.plot_tree(clf, feature_names=X.columns, class_names=['No', 'Yes'],
filled=True)
plt.show()
```

